

MULTI-USER JAVAFX WEATHER FORECASTING & IDENTITY PERSISTENCE APPLICATION

Executive Summary & Technical Architecture Report

Institution: Istanbul Topkapi University	Department: Computer Engineering	Course: Programming II
Student Name: Saliha KAPO	Student ID: 25040102005	Date: May 2026

1. EXECUTIVE SUMMARY & PROJECT OVERVIEW

This executive technical document provides a high-level summary of the cross-platform desktop Weather Forecasting Application developed using Java 11 and the JavaFX GUI framework. Engineered specifically to bridge local software security architectures with remote RESTful JSON enterprise streams, the system provides an integrated solution for authentication state containment, secure user lifecycle processing, real-time climate querying, and historic logging metrics.

The primary milestone accomplished within this architectural framework is the complete eradication of unsecured plain-text credential logging. The production tier successfully implements enterprise-level Java Object Serialization alongside cryptographic PBKDF2WithHmacSHA256 password salting models, ensuring compliance with academic standards of secure data persistence. Furthermore, the UI layer delivers a scalable dashboard parsing and rendering structural meteorologic information, including an extended multi-day predictive prognostics panel synchronized directly with the underlying application layout parameters.

2. CORE FUNCTIONAL REQUIREMENTS & HIGH-LEVEL ARCHITECTURE

The lifecycle execution is orchestrated across three distinct foundational pillars, providing distinct technical features:

- Decoupled Identity Persistence Subsystem: Handles unique user account registrations via independent controller views, enforces case-insensitive duplicates checking, executes length boundaries, and maintains secure binary object file locking.
- Real-Time Weather Data Ingestion: Synchronously establishes authenticated HTTP network loops targeting the external OpenWeatherMap REST endpoints, parsing geographical attributes and mapping conditional assets.
- User Search Tracking & Analytical Filtering: Tracks sequential search events inside an atomic shared ledger file, while actively enforcing user-specific session isolates to decouple separate profiles during runtime view binding.

3. CRYPTOGRAPHIC LAYER & OBJECT SERIALIZATION ARCHITECTURE

The persistence engine utilizes a dual-layered data storage approach, matching security requirements with transactional logging performance. The plain-text structure has been fully replaced with a secure framework:

Authentication Layer (Binary Object Store)	Search History Layer (Structured Plain Text Ledger)
File Container: users.dat Implementation: Native Java Object Serialization via ObjectOutputStream and ObjectInputStream pipelines. Security Level: Manual file manipulation breaks byte-stream patterns, triggering a StreamCorruptedException to instantly lock down the execution boundary.	File Container: history.txt Implementation: Standard buffered character streaming with append-only write privileges. Runtime Filtering: Session handles dynamically intercept, split, and sort logs using active user sub-strings, confining the UI ListView view strictly to personal data.

To neutralize credential harvesting vulnerabilities, plain-text character strings are immediately dropped upon entry. The PasswordUtil utility leverages a SecureRandom pseudo-random generator to yield a unique 16-byte salt element per account. The password string and salt undergo 120,000 computation loops using the PBKDF2WithHmacSHA256 cryptographic algorithm. During subsequent verification, the exact historical salt is unpacked from the serialized collection, ensuring a synchronized, error-immune login verification pipeline.

4. ADVANCED JSON PARSING & DYNAMIC FORECASTING DASHBOARD

Upon a validated search request, the networking tier establishes synchronous `HttpURLConnection` streams to pull remote data structures. Real-time metrics are bound to graphical icons, current descriptors, and thermal tracking labels.

The extended prognostic engine extracts structured meteorological indices from nested JSON payload arrays. The data layer slices the array payload, applying string filtering loops to group entries into single chronological blocks. The interface layer iterates across these parsed nodes to dynamically construct 5 consecutive weather forecast cards. These components are stacked inside a clean, horizontally-aligned user interface layout, ensuring a fluid, user-friendly desktop experience without visual lag.

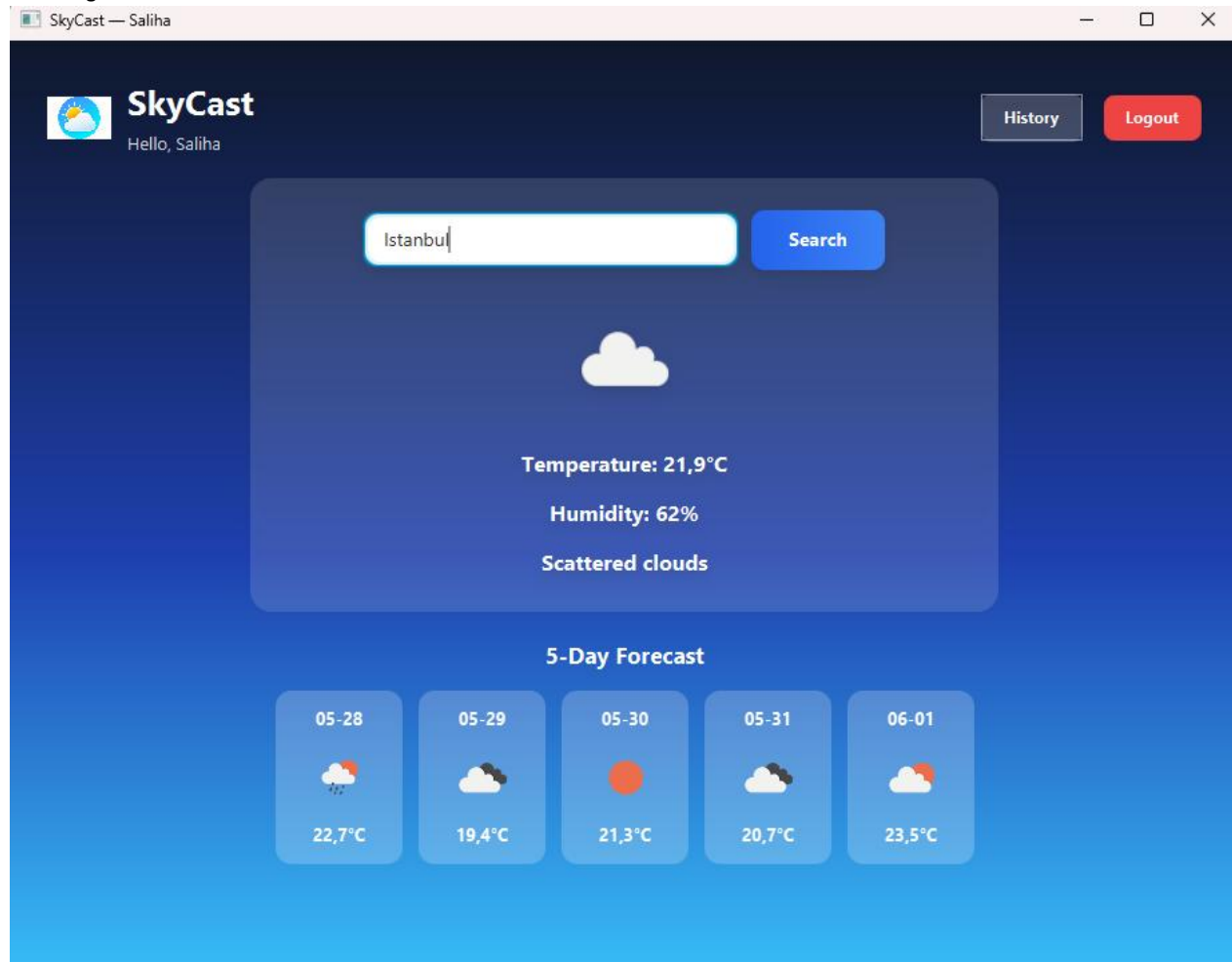


Figure-1: Responsive JavaFX Weather Application User Interface. As visualized above, the primary dashboard integrates real-time weather metrics, synchronized condition icons, dynamic individual user search history logs, and the advanced horizontal 5-day meteorological forecasting carousel.

5. PROJECT WORKFLOW VERIFICATION & ARCHITECTURAL CONCLUSION

This desktop application demonstrates practical mastery of core computer engineering paradigms: mapping dynamic network streams, securing local user persistence through object serialization, isolating logic segments via utility decoupling, and rendering flexible user interfaces with JavaFX components.

By refactoring the registration pipeline into a standalone view and implementing explicit "Password Match Validation," the architectural robustness of the system is greatly increased. The application establishes an excellent foundation for real-world production setups. Future development cycles can scale this baseline by integrating full relational database systems (such as SQLite or PostgreSQL) and incorporating automated multi-language localized resource bundles.